Description Submission view

Available from: Cuma, 17 Nisan 2026, 11:08 AM
Due date: Cuma, 17 Nisan 2026, 12:20 PM
Requested files: FactoryLogisticsSimulator.java (Download)
Type of work: Individual work
Reduction by automatic evaluation: 30 **Free evaluations**: 4**Reduction by automatic evaluation**: 30 **Free evaluations**: 4

Laboratory Manual: Tech Factory Logistics and Assembly Simulation

In this session, you will design the architecture for a technology manufacturing factory and its logistics network. This exercise is designed to reinforce your understanding of core OOP principles, including Encapsulation (Immutability), Inheritance, Abstract Classes, Interfaces, Polymorphism, Java Generics, and 2D Array manipulation alongside Java Collections (Queue and Stack).

Please read the instructions carefully. Your system will be evaluated using an automated test suite based on strict structural and mathematical requirements. The Main execution file will be provided by the testing system.

Objectives

- Build a robust class hierarchy representing different technology modules and machines.
- Implement interfaces to categorize object capabilities based on logistics and shipping.
- Manage object states and interactions as they pass through a simulated assembly grid (2D array).
- Design a custom generic storage class that enforces specific warehouse retrieval behaviors.
- Enforce strict encapsulation by making specific object properties immutable after instantiation.

Grading Rubric (Total: 100 Points)
Your submission will be evaluated based on the following automated point distribution:

- **Category 1: Architecture & Encapsulation (15.0 Points)**
  Correct class hierarchy, strict encapsulation enforcement (omission of specific setter methods), and correct implementation of Java Generics <T>: 3 tests x 5.0 pts (weighted avg) = 15.0 Points.
- **Category 2: Abstract Interface Method Promotion (20.0 Points)**
  Interface method promotion, Manhattan distance translations, and coordinate edge cases: 10 tests x 2.0 pts = 20.0 Points.
- **Category 3: Module Build Pricing (20.0 Points)**
  Accurate base cost calculations for specific tech units and getter/setter validations: 10 tests x 2.0 pts = 20.0 Points.
- **Category 4: 2D Automation Array & Null Safety (30.0 Points)**
  Robust 2D array traversal, NullPointerException prevention, Queue/Stack operations, and generic capacity bounds: 15 tests x 2.0 pts = 30.0 Points.
- **Category 5: Full Integration Reliability (15.0 Points)**
  End-to-end execution of the laboratory scenario, validating precise LIFO extraction and accumulated wiring logic: 1 test x 15.0 pts = 15.0 Points.

?

System Architecture Requirements

You are required to create the following components. Ensure that all class names, method names, and parameter types match these descriptions exactly. Do not use import statements unless specifically instructed (you may import java.util.Stack and java.util.Queue for the floor manager class).

1. The Capability Interface: IShippable

This interface defines the contract for items that can be transported to a shipping bay.

- **Structure:** Create an interface named IShippable.
- **Method:** Declare a single method named calculateShippingFee that takes two parameters (int startX, int startY) and returns a double.

2. The Data Carrier & Blueprint: TechModule

This abstract class serves as the foundational blueprint for all technology modules and implements the IShippable interface.

- **Structure:** Create an abstract class named TechModule.
- **Attributes:** Define four private fields: modelName (String), bayX (int), bayY (int), and manufacturingCost (double).
- **Constructor:** Accept parameters for modelName, bayX, and bayY. Initialize these attributes and strictly set manufacturingCost to 0.0.
- **Encapsulation (Strict):** Provide standard public getter methods for all attributes (getModelName, getBayX, getBayY, getManufacturingCost). Provide setter methods for bayX, bayY, and manufacturingCost. **Crucially, do NOT create a setModelName method.** The module's name must be strictly immutable after instantiation.
- **Behaviors:** Implements calculateShippingFee(int startX, int startY). You must use the Manhattan distance logic. Calculate the absolute difference between bayX and startX, plus the absolute difference between bayY and startY. Multiply the total distance by 15.0 to return the final shipping fee.

3. Concrete Technology Classes

You will now design the specific modules by extending your blueprint.

- **A. ServerUnit**
  - **Inheritance:** Extends TechModule.
  - **Constructor:** Accepts a String modelName, int bayX, int bayY, and int cpuCount. Passes the first three to the parent constructor.
  - **Logic:** Calculates and sets the initial manufacturing cost using the formula: 500.0 + (cpuCount * 120.0).
- **B. DroneUnit**
  - **Inheritance:** Extends TechModule.
  - **Constructor:** Accepts a String modelName, int bayX, int bayY, and double cubicFrameSize. Passes the first three to the parent constructor.
  - **Logic:** Calculates and sets the initial manufacturing cost using the formula: 100.0 + (cubicFrameSize * cubicFrameSize * cubicFrameSize * 0.2).

4. The Machine Blueprint: MachineUnit

Represents a physical automated machine on the factory floor.

- **Structure:** Create an abstract class named MachineUnit.
- **Attributes:** Define a private String field named name.

- **Constructor:** Accept a String parameter to initialize the name attribute.
- **Encapsulation (Strict):** Provide a public getter method named getName. **Do NOT create a setName method.**
- **Behaviors:** Declare a public abstract method named processHardware. This method must return void and accept a single parameter of type TechModule (name the parameter module).

5. Concrete Machine Classes

- **A. WiringMachine**
  - **Inheritance:** Extends MachineUnit.
  - **Constructor:** Accepts a String name and passes it to the parent constructor.
  - **Logic (processHardware):** This unit increases the module's current manufacturingCost by exactly 50.0 per pass. It must do this silently (no console print statements).

6. Generic Class: WarehouseBin<T>

A custom generic storage utility class that strictly enforces a bounded Last-In, First-Out (LIFO) behavior.

- **Attributes:** An array of Object type named parts to hold the generic items, and an integer count.
- **Constructor:** Accepts an integer for the capacity. Initializes the object array to this exact capacity and sets the count to 0.
- **Methods:**
  - addPart(T part): Safely adds the item to the array only if the current count is less than the maximum capacity. Crucially, if the bin is full, it must simply ignore the addition without throwing an out-of-bounds error.
  - retrievePart(): Decrements the count and returns the last added item of generic type T. If the bin is completely empty, it must safely return null. Ensure proper generic type casting.

7. The Facility Manager: FactoryFloor

This central class manages the flow of hardware through the physical grid to the warehouse.

- **Attributes:**
  - A 2D array of MachineUnit named grid.
  - A standard Java Queue<TechModule> named chassisQueue. This represents the incoming line of raw modules that must be processed in a First-In, First-Out (FIFO) manner.
  - A standard Java Stack<TechModule> named readyModules. This represents the loading dock where finished modules are placed in a Last-In, First-Out (LIFO) manner.
- **Constructor:** Accepts the 2D grid array and the chassisQueue. Initializes the readyModules Stack.
- **Method: runAssemblyLine**
  - Returns void and takes no parameters.

- While the incoming chassisQueue is not empty, extract a module using standard queue operations (poll).
- Use standard integrated nested for-loops to iterate through the 2D grid array (row by row, column by column).
- If a grid coordinate contains a machine (is not null), invoke its processHardware method on the extracted module.
- Once the module has traversed the entire grid, push it onto the readyModules Stack.

getReadyModules(): Returns the `Stack` containing the fully processed modules ready for shipping. (readyModules)

Edge Cases and Boundary Conditions

To achieve a perfect score, your system must handle the following edge cases robustly, as tested by the automated system:

- **Extreme and Zero Inputs:** Instantiating a ServerUnit with 0 CPUs or a DroneUnit with a 0.0 frame size must simply yield the starting constant base cost without failing.
- **Logistics Computations:** If a module is shipped to the exact starting coordinates (distance is 0), the fee must be exactly 0.0. Shipping destinations can exist in negative coordinate spaces (e.g., X: -3, Y: -4) or mixed alignments. Your implementation must handle absolute distances accurately utilizing Math.abs().
- **Grid Null Safety:** If the FactoryFloor is initialized with a grid filled completely or partially with null spaces, the loop must safely bypass them without throwing a NullPointerException.
- **Cumulative Processing:** The same module might pass through multiple WiringMachine units sequentially. The cost must continually accumulate on top of the current total per pass.
- **Storage Boundaries:** Your WarehouseBin<T> must safely ignore additions if it exceeds its initialized capacity, and return null gracefully if a retrieval is attempted while it is empty.

Example Scenario for Self-Verification

Before submitting, trace this mathematical scenario on paper to ensure your logic is sound:

1. Factory starts at X: 0, Y: 0.
2. A 2x2 grid is created: [0][0] is a WiringMachine, [0][1] is null, [1][0] is a WiringMachine, [1][1] is null.
3. An "AeroScout Pro" Drone (Size 5.0, Dest: 20, 25) enters the queue.
    - Initial Base Cost: 100.0 + (125.0 * 0.2) = 125.00.
    - Passes Wiring 1: +50.0 (Total: 175.00).
    - Passes Wiring 2: +50.0 (Total: 225.00).
    - Shipping Fee: (|20 - 0| + |25 - 0|) * 15.0 = 675.00.
    - Final Output Total: 225.00 + 675.00 = 900.00.

Example Output

```
--- LOGISTICS SUMMARY ---
Shipping AeroScout Pro. Total: $900,00
Shipping Enterprise Blade. Total: $1455,00
Total Value Shipped: $2355,00
```

Requested files

FactoryLogisticsSimulator.java

```java
import java.util.*;

public class FactoryLogisticsSimulator {
    public static void main(String[] args) {
        int factoryX = 0; int factoryY = 0;

        MachineUnit[][] machineGrid = new MachineUnit[2][2];
        machineGrid[0][0] = new WiringMachine("Arm Auto-Wirer");
        machineGrid[0][1] = null;
        machineGrid[1][0] = new WiringMachine("Core Solderer");
        machineGrid[1][1] = null;

        Queue<TechModule> assemblyQueue = new ArrayDeque<>();
        assemblyQueue.add(new ServerUnit("Enterprise Blade", 10, 15, 4));
        assemblyQueue.add(new DroneUnit("AeroScout Pro", 20, 25, 5.0));

        FactoryFloor floor = new FactoryFloor(machineGrid, assemblyQueue);
        floor.runAssemblyLine();

        System.out.println("--- LOGISTICS SUMMARY ---");
        Stack<TechModule> shippingStack = floor.getReadyModules();
        double totalInvoicedValue = 0.0;

        while (!shippingStack.isEmpty()) {
            TechModule module = shippingStack.pop();
            double logisticsFee = module.calculateShippingFee(factoryX, factoryY);
            double finalInvoicePrice = module.getManufacturingCost() + logisticsFee;
            totalInvoicedValue += finalInvoicePrice;

            System.out.printf("Shipping %s. Total: $%.2f\n", module.getModelName(), finalInvoicePrice);
        }
        System.out.printf("Total Value Shipped: $%.2f\n", totalInvoicedValue);
    }
}
```

VPL